Artificial Intelligence

Human-Computer Interaction

Lecture Notes

MultiModal Systems — Theory 2

ACADEMIC YEAR

2025–2026

Jacopo Parretti

Contents

Part I

Foundations of Multimodal Interaction

1 Introduction to Intelligent Multimodal Interfaces

1.1 Course Objectives

This course explores the fundamental theories and concepts of **Human-Computer Interaction (HCI)**, an interdisciplinary field that synthesizes knowledge from cognitive psychology, computer science, and design. The primary objectives are:

- Understanding the theoretical foundations of human-computer interaction
- Developing practical skills in designing and implementing multimodal interfaces
- Exploring the integration of artificial intelligence techniques in interactive systems
- Analyzing nonverbal communication and its role in human-computer interaction

1.2 Focus Areas

The course places special emphasis on three interconnected dimensions:

1.2.1 Technological Solutions

Students will develop computer interfaces with a focus on both methodological and implementation aspects. The course emphasizes hands-on experience in building functional interactive systems, bridging the gap between theory and practice.

1.2.2 Multimodal Interaction

Multimodal Input and Output Modalities

Special attention is devoted to **multimodal solutions** that integrate multiple input and output modalities:

- **Touch**: Tactile and haptic interaction
- **Vision**: Camera-based interaction and visual recognition
- **Natural Language**: Speech and text-based communication
- **Audio**: Sound-based interaction and auditory feedback

1.2.3 Intelligent Systems

The course explores how **artificial intelligence techniques** can enhance interaction by:

- Inferring user intentions from multimodal input
- Predicting expected interactions based on context and user behavior
- Adapting interface behavior to individual users
- Recognizing and responding to affective and social cues

1.3 Course Program

1.3.1 Theoretical Component

The theoretical component covers the following topics:

1. **Introduction**: Course motivation, professional perspectives, open research issues, program overview, and examination methodology
2. **Foundations of HCI**: Human factors in interface design, interaction design principles, usability evaluation, gaming, and gamification
3. **Visual Interaction**: Camera calibration techniques, structure from motion, 3D reconstruction
4. **Nonverbal Behavior in Communication**:
   - Types of nonverbal behavior: facial expressions, gestures, posture, eye gaze
   - Data collection methods and protocols
   - Tools and software for nonverbal behavior analysis
   - Annotation tools (e.g., ELAN)
5. **Automated Analysis of Body Language**: Movement tracking, gesture recognition, facial expression analysis, and speech processing. Techniques for data capture, feature extraction, and automatic analysis
6. **Social Artificial Intelligence**: Applications in social psychology, organizational psychology, and social robotics
7. **Affective Computing**: Theories of emotion, emotion recognition systems, and applications in HCI
8. **Multimodal Fusion**: Integration of multimodal nonverbal cues using fusion techniques (late fusion, early fusion)

1.3.2 Laboratory Component

The laboratory sessions provide hands-on experience with state-of-the-art tools and techniques:

1. **Deep Image Matching**: Python implementation of feature detection and matching algorithms
2. **3D Model Reconstruction**: Structure from motion using Zephyr software
3. **Camera Pose Estimation**: C# implementation of Fiore's method for camera localization
4. **3D Graphics**: Modeling and rendering in Unity game engine
5. **Model-Based Augmented Reality**: Implementation of the complete AR pipeline integrating Python code and Unity
6. **Advanced Topics**: Deep learning approaches to camera pose estimation and model recognition

2 The Evolution of Human-Computer Interaction

2.1 Why Study HCI?

The fundamental motivation for studying Human-Computer Interaction lies in understanding the **interaction space** between humans and machines. Effective interaction design must strike a balance between these two poles—neither overly machine-centric nor purely human-centric, but rather positioned at an optimal middle ground that accommodates both human capabilities and computational constraints.

The central challenge of HCI is to design interfaces that allow users to accomplish their goals efficiently and naturally, while leveraging the computational power and capabilities of modern computing systems.

2.2 Historical Evolution of Interaction Paradigms

The field of HCI has undergone several paradigm shifts, each bringing the interaction closer to natural human behavior.

2.2.1 Command-Line Interfaces (Pre-1980s)

Command-Line Interfaces (CLI)

Early human-computer interaction relied on **command-line interfaces (CLI)**, which required users to communicate with machines through text-based commands. This paradigm represented an interaction model heavily biased toward the machine:

- **Expert-oriented**: CLIs assume users possess detailed knowledge of the system's internal organization and command syntax
- **Machine-centric**: The interface reflects the machine's architecture rather than human cognitive models
- **High learning curve**: Users must memorize commands, parameters, and system-specific conventions
- **Limited accessibility**: Generic users cannot interact effectively without substantial training

While command-line interfaces persist today (particularly among expert users who value direct system control and automation capabilities), they exemplify an interaction paradigm positioned closer to the machine than to the human.

2.2.2 Graphical User Interfaces (1980s–2000s)

Graphical User Interfaces (GUIs)

The introduction of **Graphical User Interfaces (GUIs)** in the early 1980s marked a fundamental shift in interaction design:

- **Direct manipulation**: Users interact with visual representations of objects through pointing devices (mouse) and keyboards
- **Visual metaphors**: Desktop metaphors, windows, icons, and menus provide intuitive representations of system functionality
- **Reduced cognitive load**: Users no longer need to memorize complex command syntax
- **Increased accessibility**: Non-expert users can accomplish tasks through exploration and recognition rather than recall

This paradigm shift moved interaction significantly closer to human cognitive models, making computing accessible to a broader population.

2.2.3 Pervasive and Ubiquitous Interaction (2000s–Present)

Pervasive and Ubiquitous Computing

Over the past 15–20 years, a new paradigm has emerged: **pervasive interaction** (also known as ubiquitous computing). This paradigm is characterized by:

- **Distributed computing**: Intelligence is distributed across multiple interconnected devices (smartphones, wearables, IoT sensors, smart home devices)
- **Invisible interfaces**: The traditional notion of "the machine" becomes diffuse—computation is embedded in the environment
- **Context awareness**: Systems adapt to user context, location, and activity
- **Natural interaction**: Voice, gesture, and multimodal input replace traditional keyboard and mouse
- **Smart environments**: Individual devices collaborate to create intelligent, responsive spaces

In contrast to earlier paradigms where "the machine" was a discrete entity contained in a single case, pervasive computing dissolves the boundaries between user, environment, and computational infrastructure. This represents the closest approximation yet to natural human interaction patterns.

2.3 Technological Foundations

This course emphasizes the **technological aspects** that enable modern HCI systems. Implementing sophisticated, intelligent, and multimodal interactions requires substantial infrastructure:

- **Network infrastructure**: High-bandwidth internet connectivity, satellite networks, and edge computing for distributed systems
- **Interaction devices**: Advanced input/output hardware including VR/AR headsets, depth cameras, haptic devices, and multimodal sensors
- **Computational power**: GPU acceleration, cloud computing resources, and specialized AI hardware for real-time processing
- **Software frameworks**: Machine learning libraries, computer vision toolkits, and multimodal fusion platforms

Understanding these technological foundations is essential for designing and implementing the next generation of human-computer interfaces.

3 Human Factors

Human factors are the psychological and physiological characteristics of the human body that are relevant to the design of human-computer interfaces. Perceptual aspects such as vision, hearing, touch, taste, and smell are relevant to the design of human-computer interfaces. We will mainly focus on visual aspects.

3.1 Brightness perception

Brightness perception is the ability to perceive the brightness of a light.

- **Brightness** perception is subjective and depends on the individual.
- **Brightness** is conditioned by the quantity of light.
- **Brightness** is a function of the object brightness and the background brightness.
- **Visual acuity** is the ability to see details and increases with lighter scenes.

The *flicker effect* perceived in large monitors is due to the refresh rate of the monitor. A monitor with a refresh rate of 60 Hz refreshes the image 60 times per second. If the image is not refreshed at this rate, the eye will perceive a *flicker effect*.

3.2 Abilities and limitations of vision processing

The visual system compensates for motion (i.e., *image stabilization*) and light (or color) variations. Examples of overcompensations are demonstrated in the following optical illusions:

3.3 Context awareness

Context is important for the perception of the environment. For instance, the perception of a face is different in a dark room than in a bright room. Limits of vision processing are also affected by the context.

While reading we follow a few steps:

- perception of visual patterns,
- patterns are decoded using the internal representation of the language,
- they are explained using syntactic and semantic knowledge (and pragmatics)

Reading is performed by *saccadic eye movement* and *visual fixation*. The shape of a single word is very important for the perception of the word.

3.4 Cognitive aspects of vision processing

Some of the cognitive aspects of vision processing are the following:

- Long-term memory and learning
- Problem solving
- Decision making
- Attention and problem dimensionality

3.5 Human Memory

We can differentiate between different *sensorial buffers*:

- *iconic memory*
- *echoic memory*
- *haptic memory*

These sensorial buffers after being processed by the *attention mechanism* are stored in the **short-term memory**. The **short-term memory** is then processed by the **working memory** and stored in the **long-term memory** after enough repetitions.

3.6 Short-term memory (STM)

Miller's Law and Chunking

Information is remembered better when organized in chunks. The chunk organization is called *closure* and is classically summarized by *Miller's Law*. In general a user can remember 7 ± 2 chunks The user always expects a closure at the end of a task. For example, in early cash retrieval machines users often forgot to take the card after getting the cash.

3.7 Inductive reasoning

Inductive reasoning describes the generalization of unseen examples from seen examples: For example, if all cars I have seen have four wheels, then all cars have four wheels. This is not fully reliable, as we can prove falsity but not absolute truth. Humans are generally not able to use negative evidence effectively.

4 Human and Virtual Reality

Virtual reality is a technology that allows the user to interact with a virtual environment. There are many issues about human perception that must be considered in depth when analyzing interaction in VR. These issues are often neglected for classic **HCI** based on intentional actions and simple stimuli. Some of the issues are the following:

- Awareness, attention
- Information fusion
- Hierarchical processing
- Adaptation
- Psychophysics

4.1 Human senses

Humans have more than five senses. Some of these senses include:

- **Proprioception**: sense of body position—what is your body doing right now
- **Equilibrium**: balance
- **Acceleration**: perception of linear and angular changes
- **Nociception**: sense of pain
- **Thermoreception**: temperature
- **Satiety**
- **Thirst**
- **Micturition**
- **Chemoreception**: amount of CO_2 and Na^+ in blood

In virtual reality **vision** is the main channel, but other senses are also important and therefore must be coherent with the virtual environment. **Sensing** is an active process that depends on previous inputs **not in a conscious way**.

4.2 Adaptation

Adaptation is the process of changing the perception of a stimulus over time. For instance, the perceived loudness of motor noise in an aircraft or car decreases within minutes. The optical system of our eyes and the photoreceptor sensitivities adapt to change perceived brightness. Over long periods, perceptual training can lead to adaptation; In military training simulations, sickness experienced by soldiers appears to be less than expected, perhaps due to regular exposure. The same appears to be true of experienced video game players. Those who have spent many hours and days in front of large screens playing first-person shooter games apparently experience less *vection* when locomoting themselves in VR. **Adaptation** therefore becomes a crucial factor for VR.

4.3 Constraints for VR systems

In VR systems we have to consider the following constraints due to human vision:

- **Human frontal FOV**: 100°–110° horizontal per eye and up to 200°–220° with both eyes
- **Stereo overlap**: roughly 110°–120°
- **Foveal field**: approximately 60° horizontally and vertically where both eyes can see in focus

4.4 Eye movements

When designing AR/VR systems, we must take into account how human eyes move.

- **Saccades**: brisk motions lasting less than 45 ms with rotations of about 90° per second that quickly relocate the *fovea* so important features in a scene are sensed with the highest visual acuity, giving the illusion of high acuity over a large angular range. We typically have little awareness of these eye movements even though we can consciously control *fixation targets*.
- **Vestibulo-ocular reflex (VOR)**: an involuntary movement of the eyes that counteracts head motion and is one of the most important motions to understand for VR.
- **Smooth pursuit**: slow eye movement (usually $< 30°/s$) used to track a moving target feature such as a car, tennis ball, or person walking by, reducing *motion blur* on the retina.
- **Optokinetic reflex**: rapid movement of the eyes to track a fast object; the eyes rapidly and involuntarily choose features for tracking on the object.
- **Vergence stereopsis**: movement of the eyes to converge or diverge the pupils, providing important information about the distance of objects.
- **Microsaccades**: small, involuntary jerks (less than one degree) that trace out an erratic path and are believed to augment fixation control, reduce *perceptual fading* due to adaptation, and improve visual acuity.

4.5 Vergence and accommodation

Vergence is the movement of the eyes to converge or diverge the pupils. **Accommodation** is the adjustment of the eyes to focus on near objects. **Binocular disparity** is the difference in the image seen by the two eyes. **Retinal blur** is the blurring of the image seen by the eyes due to the finite size of the retina.

Vergence-Accommodation Mismatch in VR

In the real world, *vergence* and *accommodation* match, but in VR, *vergence* and *accommodation* can mismatch. This can cause discomfort:

- **fatigue**
- **nausea**
- **eye strain**
- **motion sickness**

Flicker fusion rate is the frequency at which the eyes can perceive a flickering image. It is important because it affects the perception of the image and can cause discomfort. For instance, if the flicker fusion rate is too low, the eyes will perceive the image as flickering.
Dynamic range is the ratio of the brightest to the darkest part of the image. Human vision has a dynamic range far higher than any available display technology.

4.6 Human Vision System

The human vision system is the following:

Visual acuity	20/20 is ∼1 arc min
Field of view	∼200° monocular, ∼120° binocular, ∼135° vertical
Resolution of eye	∼576 megapixels
Temporal resolution	∼60 Hz (depends on contrast, luminance)
Dynamic range	instantaneous 6.5 f-stops, adapts to 46.5 f-stops
Colour	everything in CIE xy diagram
Depth cues in 3D displays	vergence, focus, (dis)comfort
Accommodation range	∼8 cm to ∞, degrades with age

Table 1: Human Vision System

Modern displays can't exactly match this level of performance. Let's see a comparison between the human eyes and the HTC Vive (2016) VR headset:

	Human Eyes	**HTC Vive**
FOV	200° × 135°	110° × 110°
Stereo Overlap	120°	110°
Resolution	30,000 × 20,000	2,160 × 1,200
Pixels/inch	>2190 (100mm to screen)	456
Update	60 Hz	90 Hz

Table 2: Comparison between the human eyes and the HTC Vive (2016) VR headset

The following are some of the 3D cues that are used to create the illusion of depth:

- **Atmospheric artifacts**: fog, smoke, dust, etc.
- **Relative sizes**: objects that are closer are larger than objects that are further away.
- **Image blur**: objects that are further away are blurred.
- **Occlusion**: objects that are further away are occluded by objects that are closer.

- **Shadows**: objects that are further away have smaller shadows.
- **Texture**: objects that are further away have less texture.
- **Colour**: bluish objects are more distant.
- **Motion parallax**: Objects in relative motion provide depth information; closer objects appear to move faster than distant objects when the observer moves.

4.7 Issues for VR

The following are some of the issues for VR:

- **Coherence of depth cues**: Achieving coherent depth cues across all visual elements is difficult.
- **Vergence-accommodation mismatch in VR headsets**: In conventional VR displays, vergence and accommodation are decoupled because the display is at a fixed focal distance while the eyes converge at different distances. This mismatch can cause discomfort and is a fundamental limitation of current VR technology.
- **Imperfect head tracking**: If there is significant latency, visual stimuli will not appear in the correct place at the expected time, causing spatial misalignment and potential motion sickness.
- **Limited tracking systems**: Many tracking systems monitor only head orientation, which has obvious drawbacks for accurate spatial interaction.

4.8 Motion perception

Motion detection in the human visual system depends on local processing at the *retinal level*, which is then integrated at higher *cortical levels* to create a coherent perception of motion.

- **Optical flow**: The pattern of apparent motion of objects, surfaces, and edges in a visual scene caused by the relative motion between an observer and the scene. It represents the motion field projected onto the image plane and is fundamental to understanding how we perceive movement in both real and virtual environments.
- **Apparent motion in Virtual Reality**: VR systems exploit *apparent motion* by rapidly displaying sequences of static images. The brain perceives continuous motion from discrete frames, similar to how *motion pictures* and animation work. This illusion allows VR to create convincing motion experiences despite rendering discrete frames.
- **Stroboscopic apparent motion**: A visual phenomenon where discrete images presented in rapid succession are perceived as continuous motion. This is the fundamental principle behind animation, film, and VR displays, where the brain fills in the gaps between frames to create the perception of *smooth motion*.
- **Beta and Phi effects**: Two related phenomena that explain apparent motion:
  - **Beta effect**: The perception of motion between two static images when they are presented in rapid succession with appropriate timing and spatial separation.
  - **Phi effect**: The perception of motion in a single object that appears to move between different positions, creating an illusion of continuous movement.
- **Persistence of Vision**: A traditional but somewhat misleading explanation suggesting that images persist on the retina for approximately 100 ms after the stimulus is removed.

However, modern research indicates that motion perception in film and VR is better explained by the *phi phenomenon* and *beta effect* rather than true *persistence of vision*.

- **Motion perception at low frame rates**: Interestingly, motion can be perceived even at very low frame rates (as low as 2 fps), though it appears jerky and discontinuous. This observation challenges the traditional *persistence of vision* explanation and demonstrates that motion perception is more complex than simple *retinal persistence*.

4.9 Issues in HMD

The following are some of the issues in HMD:

- **Reducing image display time**: Reducing the time each image is displayed can help minimize flickering.
- **Frame rate requirements**: Flickering appears if the frame rate is not high enough; frame rates greater than 90 fps are typically acceptable.
- **Fast pixel switching**: Fast pixel switching is needed to reduce flickering. OLED displays achieve switching times of less than 0.1 ms.

4.10 Other important senses in VR

In immersive environments **somatosensory cues** complement vision and audition by providing information about contact, pressure, body pose, and motion. The somatosensory system is a distributed network of receptors in the skin, muscles, tendons, and joints that continuously reports the state of the body. Key aspects for VR design include:

- **Haptic sensations**: The skin contains specialized cells that respond to pressure, vibration, and temperature. Haptic feedback adds realism by matching virtual contact events with tactile effects such as vibration or force.
- **Tactile channel**: Tactile receptors do not have uniform density across the body. Areas such as the tongue and fingertips contain many more mechanoreceptors, allowing finer spatial discrimination of textures and edges.
- **Mechanoreceptors**: Specialized receptors that encode different aspects of touch (high-frequency vibration, stretch, low-frequency vibration, and steady pressure, respectively). Haptic devices stimulate these receptors indirectly through actuators embedded in controllers, gloves, or suits.
- **Proprioception (joint position sense)**: Sensors embedded in muscles, tendons, and joints report limb positions even with eyes closed. This ability lets users reach or touch virtual objects with confidence. Joints closer to the torso are generally sensed more accurately, and healthy adults localize hand position within $\sim$8 cm without visual feedback.
- **Kinaesthesia (joint motion sense)**: Movement is detected through dynamic responses of muscle spindles and tendon organs. Accurate motion cues help the user perceive forces and resistances generated by the virtual world.

4.10.1 Two-point discrimination

The *two-point test* measures tactile acuity by determining the minimum separation at which two simultaneous touches are perceived as distinct. Lower values indicate higher spatial sensitivity. Typical thresholds are summarized in Table 3.

Body site	Threshold distance (mm)
Finger	2–3
Cheek	6
Nose	7
Palm	10
Forehead	15
Foot	20
Belly	30
Forearm	35
Upper arm	39
Back	39
Shoulder	41
Thigh	42
Calf	45

Table 3: Approximate two-point discrimination thresholds across the body.

4.11 Proprioception and Kinesthesia: Advanced Aspects

4.11.1 Efference Copies and Motor Control

The *motor cortex*, which controls body movement, sends signals called **efference copies** to other parts of the brain to communicate which movements have been executed. This mechanism is analogous to having encoders on robot joints or wheels to indicate how much they have moved.

Implication: You cannot tickle yourself because your brain knows where your fingers are moving through these efference copies, which cancel out the expected sensory feedback.

4.12 The Vestibular System

The **vestibular system**, located in the inner ear alongside the *cochlea*, provides crucial information about balance, spatial orientation, and motion:

- **Utricle and saccule**: Together form the **otolithic system**, which measures linear acceleration. *Otoliths* (calcium carbonate crystals) move over *hair cells* to detect gravity and linear acceleration, providing information about head position relative to gravity.
- **Semicircular canals**: Three fluid-filled canals oriented in orthogonal planes measure angular acceleration (rotational head movements). Each canal is sensitive to rotation in its specific plane.

4.13 Auditory Perception

Similar to vision, auditory experiences are often surprising and based on adaptation, missing data, and assumptions filled by *neural structures*. Many interesting aspects derive from *psychophysical measurements*.

4.13.1 Auditory Illusions

- **Precedence effect**: When two nearly identical sounds arrive at slightly different times, we perceive a single sound. This is fundamental for handling **reverberation** (delayed copies of sound due to reflections), allowing us to perceive a single sound source based on the first arrival (usually the strongest). An *echo* is perceived if the temporal difference exceeds the "*echo threshold*."

- **McGurk effect**: Demonstrates the power of multisensory integration by mixing visual and auditory cues. In experiments where a video of a person speaking is dubbed with mismatched audio (e.g., hearing "ba" while seeing "ga"), most subjects perceive "da." The brain "fuses" the two inputs into a third plausible result. While not directly relevant to VR, this effect illustrates how sensory fusion works.

4.13.2 Loudness Perception

Loudness perception varies with frequency. This must be considered in audio interaction experiments, referencing **equal-loudness contours** (also known as *Fletcher-Munson curves*), which show how perceived loudness changes across different frequencies at the same *sound pressure level*.

4.13.3 Steven's Power Law

The response to a stimulus (*perceived magnitude*) is not linear but follows a *power law*:

$$p = k \cdot m^x \tag{1}$$

where p is the perceived magnitude, m is the physical magnitude, k is a constant, and x is an exponent that depends on the stimulus type (e.g., $x = 3.5$ for electric shock, $x = 0.33$ for brightness).

4.13.4 Just Noticeable Difference (JND)

The **Just Noticeable Difference (JND)** is the minimum amount by which a stimulus must be modified for a subject to perceive the change at least 50% of the time. This threshold varies across sensory modalities and stimulus types.

4.13.5 Sound Localization

Sound localization is the estimation of a sound source's position through hearing. This is crucial for VR: auditory and visual stimuli must correspond (e.g., a voice should appear to come from the avatar's mouth).

The JND applied to localization is the **Minimum Audible Angle (MAA)**, the minimum angular variation that can be detected.

4.13.6 Monaural cues (one ear)

- **Pinna (auricle)**: The asymmetric shape of the outer ear distorts incoming sound in a direction-dependent manner, especially for elevation perception.
- **Amplitude**: Decreases quadratically with distance.
- **Spectral distortion**: At distance, high frequencies attenuate more rapidly (e.g., distant vs. nearby thunder).
- **Reverberation**: Can be used for echolocation.

4.13.7 Binaural cues (two ears)

- **Interaural Level Difference (ILD)**: The difference in sound magnitude (volume) as heard by each ear. The farther ear is in acoustic "shadow" (works better for high frequencies).

- **Interaural Time Difference (ITD)**: The difference in arrival time of sound at the two ears (approximately 21.5 cm apart, resulting in a maximum delay of ∼0.6 ms).
- **Cone of confusion**: A set of positions in space that produce the same ILD and ITD, making localization ambiguous (e.g., a sound in front or behind). Head movement helps resolve this ambiguity.

4.14 Hierarchical Processing

Stimuli captured by receptors ascend through a *hierarchical network of neurons*. In early stages, signals are combined and propagated upward. In later stages, information flows bidirectionally, allowing *top-down influences* to modulate *lower-level processing*.

4.15 Information Fusion

Multisensory Integration and Sensory Conflict

Signals from multiple senses and proprioception are processed and combined by the brain with our experiences. We expect coherent information across modalities. VR often cannot provide all sensations correlated with a simulation, leading to **sensory conflict**.
This conflict can cause problems such as dizziness or nausea, particularly in cases of **vection**.

4.15.1 Vection and Optical Flow

Vection: The Illusion of Self-Motion

Vection is the illusion of self-motion. A classic example: you are stationary in a train, and the train on the adjacent track moves, giving you the illusion that you are moving backward.
In VR, vision (*optical flow*) tells the brain that you are accelerating, but the *vestibular system* (balance) and *proprioception* indicate that you are stationary. This conflict is a primary cause of **VR sickness**.
Movement is perceived (also in computer vision) by estimating "optical flow" from image sequences. Flow vectors are used to recover the type of movement (vection). Types of vection/movement include: (a) yaw, (b) pitch, (c) roll, (d) lateral, (e) vertical, (f) forward/backward.

4.15.2 Factors affecting vection sensitivity

- **Percentage of visual field (FOV)**: The larger the moving area, the greater the vection. A small moving object is perceived as "object in motion," while an entire moving environment is perceived as "I am moving."
- **Distance from center of view**: Peripheral motion contributes more to vection perception.
- **Exposure time**: Self-motion perception increases with time.
- **Spatial frequency**: Greater detail (texture) increases optical flow perception.
- **Contrast**: Higher contrast increases the optical flow signal.
- **Other sensory cues**: If coherent (e.g., engine noise, vibrations), they can enhance (correct or incorrect) motion perception.

- **Prior knowledge**: Knowing what is about to happen can alter perception.
- **Attention**: If the user is distracted (e.g., aiming with a weapon, selecting a menu), vection and its side effects can be mitigated.
- **Training/prior adaptation**: With sufficient exposure, the body can learn to distinguish vection from real movement, reducing VR sickness.

5 Interaction and Usability

5.1 Interaction Design

- **Human Factors**: The study of human capabilities and limitations in the design of human-computer interfaces.
- **Appearance and presentation**: The design of the user interface to make it attractive and easy to use.
- **Interaction**: The design of the interaction between a user and a system.

5.1.1 Human Factors

Human factors are the psychological and physiological characteristics of the human body that are relevant to the design of human-computer interfaces.

We need to identify the capabilities and limitations involved in the user activities to solve an interaction task. In particular we need to focus on the following aspects:

- **Perception**
- **Memory**
- **Attention and Learning**

We also need to give control to the user and reduce the amount of *cognitive load*, i.e. the amount of information that the user needs to process to solve an *interaction task*.

5.1.2 User Control

User control is the ability of the user to control the system. We need to give the user control and guidance over the system. The user is the one that decides which functions to use and when to use them. The interface should differentiate between different users, expertise levels and tasks building a **flexible interface**.

5.1.3 Reducing Cognitive Load

Recognition vs. Recall

It is easier for a user to *recognize* rather than *remember*. It is good practice to show the user all the available commands and options, show proper information to complete a task and organize the information to help the user focus on the *task at hand*.

5.2 Appearance and Presentation

We need to distinguish between 3 different principles:

1. Create an attractive aspect.

2. Use meaningful and recognizable object representations.
3. Keep the interface simple and consistent.

5.2.1 Create an attractive aspect

- All elements on the screen should be visible and recognizable.
- A proper use of contrast
- A proper use of colors, shapes and sizes.

5.2.2 Use meaningful and recognizable object representations

- Use clear and distinguishable objects.
- Group hierarchically the interface elements.
- A visual coherence should be achieved.

Functional Consistency

We need to obtain *functional consistency*, i.e. the same function should be represented by the same object in the same way. A consistent design provides a more *predictable* and *intuitive* interface and thus it is easier to learn for the user. Icons should suggest how to use the interface. The use of non-consistent icons is a source of *confusion* and *error* for the user.

5.3 Interaction

It is the way the user interacts with the interface. This can be achieved through the use of mouse, keyboard, touch screen, voice, etc.

Core Interaction Design Principles

We distinguish between 3 main different design principles:

- **Direct handling**: The user can directly handle the interface elements. The aim is to mimic the *natural way* of interacting with the world.
- **Feedback**: The user should receive *feedback* from the interface. This feedback should be *immediate* and *clear*. This also improves the user *awareness* and *learning*. Some examples of feedback are: highlighting a selected object, make a sound when an action is completed, etc.
- **Error tolerance**: The user should be able to recover from *errors*. The system can accept wrong actions without negative consequences.

There are also other design principles that are important to consider such as:

- **Affordance**: help the user to understand the interface and the *actions* that can be performed.
- **Causality**: help the user to understand the relationship between the actions and the results. If false causality occurs, we get an effect called *overshooting*.
- **Constraints**: help the user to understand the *limitations* of the interface.
- **Conceptual model**:

Conceptual Model

Help the user to understand the interface and the actions that can be performed. This is a *mental model* of the interface that the user can use to understand the actions that can be performed. A good conceptual model enables the user to predict the results of the actions that can be performed and help him understand relations between the device control and outputs. A bad conceptual model can lead to confusion and error for the user, it enforces the user to operate blindly and makes the identification of action effects more difficult.

- **Visibility**: it is achieved by placing the controls in a *high visibility area*. Just by looking at the interface the user can see the controls and the actions. Visibility is often violated in order to make things look more appealing.
- **Mapping**: it is a relation between two objects. It is important that this is easy to learn and remember. Some types of mappings are *spatial* and *perceptual*.

5.4 Classic Interaction and pervasive interfaces

Classic interaction is the traditional way of interacting with the interface. Traditional computer interfaces are based on the *Desktop concept*, as an example of *remediation*. Together with the Desktop concept we have the icons, mouse, menus, etc.

5.4.1 Pervasive interfaces

In pervasive interfaces, the computational infrastructure is distributed in the environment. The interface identifies and serves the user needs. Other similar concepts are *ubiquitous computing*, the *invisible computer*, the *disappearing computer*, *ambient intelligence*, and others. Computation is part of our everyday life, standard objects became *computational objects*.

- **Calm Computing**: the computer does not absorb the user's full attention, but requires an intermittent level of attention.

We need to put some limits to this **Pervasive Computing**, in order to avoid the user to be overwhelmed by the amount of information and the complexity of the interface. We also need to guarantee the user **privacy** and **security**.

Implicit Interaction in Pervasive Computing

In pervasive computing the user interaction with the machine changes:

- **Implicit input**: human *actions* and *behaviors* are captured, recognized and interpreted by the machine as inputs.
- **Implicit output**: the machine generates outputs that are not directly visible but are *integrated in the environment*.

Some examples of pervasive interfaces are:

- **wearable devices**: smartwatches, smartglasses, smartrings, etc.
- **tangible computing**: computation in everyday objects.
- **Locative media**: media and information that are tied to a *specific location*.
- **Intelligent Environments**: the computer is part of the environment and it is *integrated* in it.

Natural interfaces are more effective for the user since the user is focused on the solution of the task at hand rather than understanding the interface. The idea of pervasive interaction is nowadays feasible with the development of the **Internet of Things (IoT)** and **Artificial Intelligence (AI)**. The HCI solution must exploit *AI* and *machine learning* techniques to understand the user's needs and to provide the user with the best possible experience.

6 Feature Matching

Feature matching is the process of finding the best match between two sets of features. In images, this means finding the best matching keypoints between two sets. To correctly understand this chapter, we need to know the concept of **convolution**.

6.1 Convolution

Convolution Operation

Convolution is a fundamental mathematical operation used extensively in image processing. It combines two functions to produce a third function that expresses how the shape of one is modified by the other.

Convolution is a mathematical operation between two functions defined as follows:

$$(f * g)(t) = \int_0^t f(\tau)g(t-\tau)d\tau \quad \text{for } f, g : [0, \infty] \to \mathbb{R} \tag{2}$$

where f and g are the two functions to convolve. The meaning of this operation is that the output is the area under the product of the two functions, where one function is flipped and shifted.

In images, convolution is used to apply a filter to the image. The filter is a small matrix of weights (also called a kernel) that is applied to the image. The filter is applied by sliding it over the image and computing the dot product of the filter and the image patch at each position. The result is a new image with the same size as the original image.

Common Image Filters

Some examples of filters used in image processing:

- **Blurring**: A Gaussian filter that blurs the image by averaging neighboring pixels
- **Sharpening**: A Laplacian filter that enhances edges and fine details
- **Edge detection**: A Sobel filter that detects edges by computing gradient magnitudes

Convolution in images is crucial to build image filters for the estimation of corresponding points between two images.

6.2 Corresponding Point Computation

Feature Matching Pipeline

The idea consists in identifying a set of salient points (feature points) to be used for matching. The process involves three main phases:

1. **Salient point detection**: Identification of distinctive points in the image that can be reliably detected
2. **Salient point description**: Creation of a descriptor that uniquely characterizes each detected point
3. **Salient point matching**: Comparison and matching of descriptors between two images to find corresponding points

6.2.1 SIFT (Scale-Invariant Feature Transform)

SIFT Overview

SIFT (Scale-Invariant Feature Transform) is a powerful feature detection and description algorithm used to detect and describe salient points (keypoints) in images. The algorithm is designed to be invariant to:

- **Scale**: Features can be detected regardless of the image scale
- **Rotation**: Features remain stable under image rotation
- **Illumination**: Features are robust to changes in lighting conditions
- **Affine transformation**: Features are stable under affine transformations

The main objective of SIFT is to detect the **location** and **scale** of stable points in the image. These stable points are called **keypoints** or **feature points**, and they represent distinctive regions in the image that can be reliably detected and matched across different views.

6.2.2 Scale Space Representation

The fundamental concept behind SIFT is the **scale space**, which is a multi-scale representation of an image. The scale space is created by convolving the input image with Gaussian filters at different scales (standard deviations).

The scale space function $L(x, y, \sigma)$ is defined as:

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y) \tag{3}$$

where:

- $G(x, y, \sigma)$ is the Gaussian filter (kernel) with standard deviation σ:

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \tag{4}$$

- $I(x, y)$ is the input image
- $*$ denotes the convolution operation
- σ is the scale parameter (larger σ means more blurring)

The scale space is essentially a **stack of images**, where each image in the stack is the original image blurred by a Gaussian filter with a different scale parameter σ. As σ increases, the image becomes more blurred, effectively removing fine details and keeping only the larger structures.

6.2.3 Image Pyramid and Octaves

To efficiently compute the scale space, SIFT uses an **image pyramid** structure. The pyramid is organized into **octaves**, where each octave represents a doubling of the scale. Within each octave, multiple scale levels are computed by progressively increasing σ by a constant factor k (typically $k = \sqrt{2}$).

Image Pyramid Structure

- **Octave**: A set of images at different scales, where the scale doubles between consecutive octaves
- **Scale levels**: Within each octave, multiple blurred versions are created with increasing σ values
- **Downsampling**: After each octave, the image is downsampled by a factor of 2 to create the next octave
- This structure allows efficient computation while maintaining scale invariance

6.2.4 Difference of Gaussians (DoG)

Instead of directly computing the Laplacian of Gaussian (LoG), which is computationally expensive, SIFT uses the **Difference of Gaussians (DoG)** as an efficient approximation. The DoG is computed by subtracting two adjacent scale-space images:

$$\text{DoG}(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma) \tag{5}$$

This can be rewritten using the convolution property:

$$\text{DoG}(x, y, \sigma) = (G(x, y, k\sigma) - G(x, y, \sigma)) * I(x, y) \tag{6}$$

6.2.5 Keypoint Detection: Finding Extrema

The keypoints (salient points) in SIFT are detected as **local extrema** (maxima or minima) in the DoG scale space. This means we look for points that are either brighter or darker than all their neighbors in both spatial and scale dimensions.

Extrema Detection Process

To detect a keypoint at position (x, y) and scale σ:

1. Compare the pixel value at (x, y, σ) in the DoG with its **26 neighbors**:
   - 8 neighbors in the same scale level (spatial neighbors)
   - 9 neighbors in the scale level above $(\sigma \cdot k)$
   - 9 neighbors in the scale level below (σ / k)
2. A point is a keypoint candidate if it is either:
   - A **local maximum**: greater than all 26 neighbors, or
   - A **local minimum**: smaller than all 26 neighbors
3. This creates a **3D search space** (2D spatial + 1D scale), ensuring that keypoints are stable across both position and scale

The 3D neighborhood check ensures that detected keypoints are:

- **Spatially stable**: The point is distinctive in its local spatial neighborhood
- **Scale stable**: The point persists across different scales, making it robust to scale changes
- **Distinctive**: Only truly salient features are selected, reducing false positives

6.2.6 Stability and Robustness

The stability of a keypoint is determined by analyzing its behavior in the scale space. Points that remain extrema across multiple scales are considered more stable and reliable. The Hessian matrix of the DoG can be used to further analyze the local structure and filter out unstable keypoints (e.g., those along edges, which are less distinctive than corner-like features).

The key insight is that **stable keypoints** are those that:

- Remain extrema across multiple scale levels
- Have sufficient contrast (not too weak)
- Are not located on edges (which are less distinctive than corners or blobs)

These stable keypoints form the basis for feature matching, as they can be reliably detected and matched across different images, even when the images are taken from different viewpoints, scales, or under different lighting conditions.

6.2.7 Keypoint Orientation Assignment

After detecting keypoints, we need to assign an **orientation** to each keypoint to achieve rotation invariance. This is done by computing the gradient magnitude and direction in a local neighborhood around each keypoint.

The orientation is computed using the Gaussian-smoothed image at the scale where the keypoint was detected (not the DoG image). For each keypoint, we consider a local region (typically 16×16 pixels) around it and compute:

- **Gradient magnitude**: $m(x, y) = \sqrt{(L(x+1, y) - L(x-1, y))^2 + (L(x, y+1) - L(x, y-1))^2}$
- **Gradient direction**: $\theta(x, y) = \arctan((L(x, y+1) - L(x, y-1))/(L(x+1, y) - L(x-1, y)))$

where $L(x, y)$ is the Gaussian-smoothed image at the keypoint's scale.

A **histogram of gradient orientations** is then created from this local region, with 36 bins covering 360 degrees. The dominant orientation (the peak of the histogram) is assigned to the keypoint. If there are multiple peaks above 80% of the maximum peak, multiple keypoints are created at the same location with different orientations.

Orientation Assignment

- Computed from the Gaussian-smoothed image (not DoG) at the keypoint's scale
- Uses a 16×16 pixel local region around the keypoint
- Creates a 36-bin histogram of gradient orientations
- Assigns the dominant orientation (histogram peak) to the keypoint
- Multiple orientations can be assigned if there are secondary peaks above 80% of the maximum

6.2.8 SIFT Descriptor

The **SIFT descriptor** is a 128-dimensional vector that uniquely describes the local appearance around each keypoint. The descriptor is designed to be robust to illumination changes, small geometric distortions, and viewpoint variations.

The descriptor is computed as follows:

1. **Local region**: A 16×16 pixel region around the keypoint is considered, rotated according to the keypoint's orientation to achieve rotation invariance.

2. **Subdivision**: This region is divided into 4×4 sub-regions (16 sub-regions total), each containing 4×4 pixels.
3. **Gradient histogram**: For each 4×4 sub-region, an 8-bin histogram of gradient orientations is computed. The gradient magnitudes are weighted by a Gaussian window centered on the keypoint to give more weight to pixels closer to the keypoint.
4. **Concatenation**: The 16 histograms (one per sub-region), each with 8 bins, are concatenated to form a 128-dimensional vector ($16 \times 8 = 128$).
5. **Normalization**: The descriptor vector is normalized to unit length to achieve illumination invariance. To reduce the effect of non-linear illumination changes, values are thresholded (typically at 0.2) and renormalized.

SIFT Descriptor Properties

- **Dimensionality**: 128 elements (16 sub-regions × 8 orientation bins)
- **Rotation invariant**: The region is rotated according to keypoint orientation
- **Illumination robust**: Normalized to unit length and thresholded
- **Geometrically stable**: Uses Gaussian weighting to emphasize central pixels
- **Distinctive**: Captures local gradient patterns that uniquely identify the feature

6.2.9 Feature Matching

Once descriptors are computed for keypoints in both images, matching is performed by computing the distance between descriptor vectors. The most common distance metric is the **Euclidean distance** (L2 norm) between the 128-dimensional vectors.

For each keypoint in the first image, we find its nearest neighbor (smallest distance) and second nearest neighbor in the second image. A match is accepted if the ratio of these distances is below a threshold (typically 0.8). This helps eliminate ambiguous matches and improves matching reliability.

Nowadays, there are many new methods for feature matching obtained from deep learning approaches (such as SuperPoint, LoFtr...) which can provide better performance in certain scenarios.

7 Feature Matching with Deep Learning

Traditional feature matching methods like SIFT rely on hand-crafted features and descriptors. Deep learning approaches have revolutionized feature matching by learning optimal feature representations directly from data, often achieving superior performance in challenging scenarios such as low-texture regions, extreme viewpoint changes, and varying illumination conditions.

Advantages of Deep Learning for Feature Matching

- **Learned representations**: Features are learned from data rather than hand-designed
- **Better generalization**: Can handle challenging scenarios better than traditional methods
- **End-to-end training**: Can optimize the entire pipeline jointly
- **Context awareness**: Can leverage semantic and contextual information
- **Robustness**: Often more robust to illumination, viewpoint, and scale changes

7.1 SuperPoint

SuperPoint is a self-supervised framework for training interest point detectors and descriptors. It uses a fully convolutional neural network that operates on full-sized images and simultaneously computes pixel-level interest point locations and associated descriptors in a single forward pass.

7.1.1 Architecture

The SuperPoint architecture consists of:

- A **shared encoder** that processes the input image to extract features
- A **detector head** that predicts interest point locations and their scores
- A **descriptor head** that computes dense descriptors at each pixel location

The network outputs:

- A **heatmap** indicating the probability of an interest point at each pixel
- A **descriptor map** with 256-dimensional descriptors for each pixel

7.1.2 Self-Supervised Training

SuperPoint uses a self-supervised training approach:

1. **Synthetic pre-training**: The network is first pre-trained on synthetic geometric shapes (MagicPoint) to learn basic interest point detection
2. **Homographic adaptation**: Real images are warped with random homographies, and the network learns to detect points that are consistent across these transformations
3. **Descriptor learning**: Descriptors are trained to be similar for corresponding points and dissimilar for non-corresponding points

SuperPoint Key Features

- **Single forward pass**: Detects keypoints and computes descriptors simultaneously
- **Dense descriptors**: Provides descriptors at every pixel, not just at detected keypoints
- **Real-time capable**: Fast enough for real-time applications
- **Self-supervised**: Does not require manual annotation of correspondences

7.2 LoFTR (Loosely-coupled Feature Transform)

LoFTR (Loosely-coupled Feature Transform) is a detector-free method for local feature matching that revolutionized dense correspondence estimation. Unlike traditional methods that follow a detect-then-describe pipeline (first detect keypoints, then compute descriptors), LoFTR directly establishes correspondences between image pairs without explicit keypoint detection.

LoFTR Core Concept

LoFTR treats feature matching as a **set-to-set matching problem** rather than a point-to-point matching problem. Instead of detecting sparse keypoints, it operates on dense feature maps and uses transformer attention to establish correspondences, making it particularly effective in low-texture regions where traditional methods struggle.

7.2.1 Key Innovation

LoFTR's main innovation lies in the combination of:

- **Transformer architecture**: Uses self-attention and cross-attention mechanisms to model long-range dependencies and establish correspondences
- **Coarse-to-fine strategy**: First matches at low resolution (computationally efficient), then refines at higher resolution (high precision)
- **Detector-free approach**: Eliminates the need for explicit keypoint detection, enabling dense matching even in textureless regions
- **Positional encoding**: Incorporates geometric information directly into the feature representation

7.2.2 Architecture Overview

The LoFTR pipeline consists of four main stages:

1. **Local feature extraction**
2. **Positional encoding**
3. **LoFTR module (transformer)**
4. **Coarse-to-fine matching**

7.2.3 Stage 1: Local Feature Extraction

A CNN backbone (typically ResNet or FPN - Feature Pyramid Network) extracts dense feature maps from both input images. The backbone produces feature maps at multiple scales:

- **Coarse level**: Low-resolution feature maps (typically 1/8 of the original image size) for initial matching
- **Fine level**: High-resolution feature maps (typically 1/2 of the original image size) for refinement

The feature maps are flattened into sequences of feature vectors, where each vector corresponds to a spatial location in the image.

7.2.4 Stage 2: Positional Encoding

Since transformers are permutation-invariant, positional information must be explicitly added. LoFTR uses a **2D sinusoidal positional encoding** that encodes both the row and column positions of each feature. This encoding allows the transformer to understand the spatial relationships between features.

7.2.5 Stage 3: LoFTR Module (Transformer)

The core of LoFTR is a transformer-based module that processes the feature sequences from both images. The module consists of multiple layers, each containing:

- **Self-attention**: Each image attends to its own features, allowing the model to understand local context and relationships within each image
- **Cross-attention**: Features from one image attend to features from the other image, establishing correspondences between the two images

- **Feed-forward network**: A standard MLP that processes the attended features

The attention mechanism computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where:

- Q (Query), K (Key), V (Value) are learned linear projections of the input features
- d_k is the dimension of the key vectors
- The softmax normalizes attention weights

The transformer iteratively refines the features through multiple layers, progressively improving the correspondence quality.

7.2.6 Stage 4: Coarse-to-Fine Matching

LoFTR uses a two-stage matching strategy:

Coarse Matching:

- Operates on low-resolution feature maps (1/8 scale)
- Computes a **similarity matrix** between all feature pairs from the two images
- Uses **dual-softmax** operation to compute matching probabilities:

$$P(i \leftrightarrow j) = \text{softmax}(S_{i,\cdot})_j \cdot \text{softmax}(S_{\cdot,j})_i$$

  where S is the similarity matrix and $P(i \leftrightarrow j)$ is the probability that location i in image 1 matches location j in image 2
- Selects matches with high confidence (mutual nearest neighbors)

Fine Matching:

- For each coarse match, extracts a local patch from the high-resolution feature maps
- Applies the LoFTR module again on these local patches to refine the correspondence
- Achieves sub-pixel accuracy by interpolating the matching location

Coarse-to-Fine Strategy Benefits

- **Efficiency**: Coarse matching reduces the search space from $H \times W \times H \times W$ to a much smaller set of candidate matches
- **Accuracy**: Fine matching refines correspondences to sub-pixel precision
- **Robustness**: Coarse matching provides good initialization even in challenging scenarios
- **Scalability**: Can handle high-resolution images efficiently

7.2.7 Training

LoFTR is trained in a supervised manner using ground-truth correspondences from datasets like MegaDepth or ScanNet. The training loss combines:

- **Matching loss**: Encourages correct correspondences

- **Confidence loss**: Ensures high confidence for correct matches and low confidence for incorrect ones

The model learns to:

- Extract discriminative features that are robust to viewpoint and illumination changes
- Use attention to focus on relevant regions for matching
- Establish correspondences even in challenging scenarios (low texture, occlusions, etc.)

7.2.8 Advantages

LoFTR Advantages

- **Detector-free**: No need for explicit keypoint detection, enabling matching in textureless regions
- **Dense matching**: Can establish correspondences at every pixel location, not just at detected keypoints
- **Global context**: Transformer attention mechanism captures long-range dependencies and geometric relationships
- **Robust to viewpoint changes**: Handles large viewpoint differences (up to 60° rotation) better than local methods
- **Sub-pixel accuracy**: Achieves sub-pixel matching precision through fine-level refinement
- **Handles low-texture regions**: Particularly effective in areas with repetitive patterns or uniform textures
- **End-to-end trainable**: The entire pipeline can be optimized jointly

Despite its strengths, LoFTR has some limitations:

- **Computational cost**: Transformer attention is computationally expensive, especially for high-resolution images
- **Memory requirements**: Dense feature maps require significant memory
- **Training data**: Requires large amounts of training data with ground-truth correspondences
- **Real-time performance**: May be slower than traditional methods for real-time applications

7.2.9 Applications

LoFTR has been successfully applied to:

- **Visual localization**: Camera pose estimation from images
- **Structure from Motion**: 3D reconstruction from image sequences
- **Image matching**: Finding correspondences for image stitching and panorama creation
- **SLAM**: Visual SLAM systems that require robust feature matching
- **Augmented Reality**: Camera tracking and scene understanding

7.3 Comparison with Traditional Methods

Deep Learning vs. Traditional Methods

Aspect	Traditional (SIFT)	Deep Learning
Feature design	Hand-crafted	Learned from data
Keypoint detection	Explicit (DoG extrema)	Implicit or learned
Descriptor	Fixed 128-dim vector	Learned, variable size
Training	No training needed	Requires training data
Generalization	Limited	Better on diverse data
Low-texture regions	Struggles	Often better
Computational cost	Moderate	Can be higher
Real-time capability	Yes	Depends on architecture

7.4 Applications

Deep learning-based feature matching has found applications in:

- **Visual SLAM**: Simultaneous Localization and Mapping using visual features
- **Structure from Motion**: 3D reconstruction from image sequences
- **Image stitching**: Panorama creation
- **Augmented Reality**: Camera pose estimation and tracking
- **Visual odometry**: Motion estimation from camera images
- **Object tracking**: Tracking objects across frames

8 Fundamentals of Computer Vision

Computer vision is the field of study that focuses on enabling computers to interpret and understand visual information from the world. It involves extracting meaningful information from images and videos, which is fundamental for many HCI applications, including augmented reality, gesture recognition, and visual tracking.

8.1 Image Acquisition and Representation

This section introduces how the physical world is converted into digital data. Understanding the image acquisition process is fundamental to computer vision.

8.1.1 Digital Images

Digital images are defined as **2D arrays** (matrices) of numbers that represent the light intensity recorded by a photosensitive device. Each element in the array is called a **pixel** (picture element).

Digital Image Representation

- Images are stored as 2D matrices $I(x, y)$ where (x, y) are pixel coordinates
- Each pixel value represents light intensity (brightness) at that location
- For grayscale images: single value per pixel (typically 0-255 for 8-bit images)
- For color images: multiple values per pixel (e.g., RGB with red, green, blue channels)

8.1.2 Optical Process

The optical process describes the path of light rays through the camera system:

- **Lenses**: Light rays from the 3D world pass through camera lenses, which focus the light
- **Aperture (pupilla)**: The opening that controls how much light enters the camera
- **Image plane**: The surface where the image is formed
- **Color filters**: Modern cameras use **Bayer pattern** filters on sensors to capture color information. The Bayer pattern arranges red, green, and blue filters in a specific mosaic pattern
- **Infrared filters**: Many cameras include IR filters to block infrared light and improve color accuracy

8.1.3 Focus and Aperture

The concept of **focus** is crucial for image formation:

Point in Focus

For a point to be in focus, all light rays from a 3D point **P** must converge to a single image point **p** on the image plane.

Aperture and Sharpness:

- By reducing the aperture to a point (pinhole), we obtain a one-to-one correspondence between visible points and image points
- This increases sharpness (all points are in focus) but reduces the amount of light entering the camera
- Larger apertures allow more light but create depth-of-field effects (only points at a specific distance are in focus)

8.1.4 Sensors

The image is discretized using photosensitive sensors:

- **CCD (Charge-Coupled Device)**: Converts light energy into electrical charge, which is then read out and converted to digital values
- **CMOS (Complementary Metal-Oxide-Semiconductor)**: Each pixel has its own amplifier, allowing faster readout and lower power consumption

The conversion process:

1. Light photons hit the sensor, generating electrical charge proportional to light intensity
2. The charge is converted to voltage
3. The voltage is quantized into discrete integer values (e.g., 0-255 for 8-bit images)
4. These values form the digital image matrix

8.2 Image Geometry (Pinhole Model)

This section defines the mathematical model that relates 3D world coordinates to 2D image coordinates.

8.2.1 Camera Obscura

The simplest geometric model is the **pinhole camera**, derived from the Renaissance principle of the *camera obscura*. In this model:

- The camera has an infinitesimally small aperture (pinhole)
- All light rays pass through a single point (the optical center)
- The image is formed on a plane behind the pinhole
- There is a one-to-one correspondence between points in the 3D world and points on the image plane

8.2.2 Perspective Projection Equations

Given a 3D point $\mathbf{M} = [X, Y, Z]^T$ in the world coordinate system and its projection $\mathbf{m} = [u, v]^T$ on the image plane, the perspective projection equations are:

$$u = -f\frac{X}{Z}, \quad v = -f\frac{Y}{Z} \tag{7}$$

where:

- f represents the **focal length** (distance between the center of projection and the image plane)
- The negative sign indicates that the image is inverted (or we can place the image plane in front of the pinhole to avoid inversion)
- These relations are **non-linear** due to the division by the depth Z

Perspective Projection Properties

- Points farther from the camera appear smaller (perspective foreshortening)
- Parallel lines in the world converge to a vanishing point in the image
- The projection depends on the depth Z, making it a non-linear transformation

8.2.3 Homogeneous Coordinates

By introducing **homogeneous coordinates**, the projection becomes a linear operation. A 3D point $\mathbf{M} = [X, Y, Z]^T$ is represented in homogeneous coordinates as $\mathbf{M} = [X, Y, Z, 1]^T$.

The projection can then be written as:

$$\mathbf{m} \simeq \mathbf{PM} \tag{8}$$

where:

- $\mathbf{m} = [u, v, w]^T$ is the projection in homogeneous coordinates
- $\mathbf{P}$ is the 3×4 projection matrix
- $\simeq$ means equality up to a scale factor
- The actual image coordinates are: $u_{img} = u/w$, $v_{img} = v/w$

In matrix form:

$$\begin{pmatrix} u \\ v \\ w \end{pmatrix} = \begin{pmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}$$

8.2.4 Perspective vs Orthographic

When the depth variation of objects is negligible compared to their distance from the camera ($\Delta Z/Z_0 \ll 1$), we can approximate the perspective projection with a **scaled orthographic projection** (also called **weak perspective**).

In weak perspective:

$$u = -s\frac{X}{Z_0}, \quad v = -s\frac{Y}{Z_0} \tag{9}$$

where $s = f/Z_0$ is a constant scale factor, and Z_0 is the average depth of the object.

Weak Perspective Approximation

Weak perspective is valid when:
- Objects are far from the camera relative to their size
- The depth variation within the object is small
- This approximation simplifies many computer vision algorithms

8.3 Camera Parameters

The complete mathematical model is decomposed into intrinsic and extrinsic parameters.

8.3.1 Intrinsic Parameters (Matrix K)

Intrinsic parameters (encoded in the matrix **K**) describe the internal properties of the camera and are independent of its position and orientation.

Transformation from sensor coordinates (meters) to pixel coordinates:

The intrinsic parameters include:

- **Focal length in pixels** (α_u, α_v): The focal length expressed in pixel units. If the physical focal length is f (in meters) and the pixel size is (s_x, s_y) (in meters per pixel), then:

$$\alpha_u = \frac{f}{s_x}, \quad \alpha_v = \frac{f}{s_y}$$

- **Principal point** (u_0, v_0): The optical center expressed in pixel coordinates (the point where the optical axis intersects the image plane)
- **Skew factor** s: Accounts for non-rectangular pixels (usually 0 for modern cameras)

The intrinsic matrix **K** is:

$$\mathbf{K} = \begin{pmatrix} \alpha_u & s & u_0 \\ 0 & \alpha_v & v_0 \\ 0 & 0 & 1 \end{pmatrix} \tag{10}$$

Intrinsic Parameters

- Intrinsic parameters are fixed for a given camera (unless the zoom changes)
- They transform points from the camera coordinate system to pixel coordinates
- They account for the physical properties of the sensor and lens

8.3.2 Extrinsic Parameters (R, t)

Extrinsic parameters describe the camera's pose (position and orientation) in the world coordinate system.

- **Rotation matrix R**: A 3×3 orthogonal matrix representing the camera's orientation. It transforms coordinates from the world frame to the camera frame
- **Translation vector t**: A 3×1 vector representing the camera's position (the world origin expressed in camera coordinates)

The transformation from world coordinates to camera coordinates is:

$$\mathbf{M}_{cam} = \mathbf{R}\mathbf{M}_{world} + \mathbf{t} \tag{11}$$

In homogeneous coordinates:

$$\begin{pmatrix} X_{cam} \\ Y_{cam} \\ Z_{cam} \\ 1 \end{pmatrix} = \begin{pmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{pmatrix} \begin{pmatrix} X_{world} \\ Y_{world} \\ Z_{world} \\ 1 \end{pmatrix}$$

8.3.3 Full Projection Matrix (P)

The complete projection matrix $\mathbf{P}$ combines both intrinsic and extrinsic parameters:

$$\mathbf{P} = \mathbf{K}[\mathbf{R}|\mathbf{t}] \tag{12}$$

where $[\mathbf{R}|\mathbf{t}]$ is a 3×4 matrix formed by concatenating the rotation matrix and translation vector.

The full projection equation is:

$$\mathbf{m} \simeq \mathbf{P}\mathbf{M}_{world} = \mathbf{K}[\mathbf{R}|\mathbf{t}]\mathbf{M}_{world} \tag{13}$$

This matrix has 11 degrees of freedom (12 elements minus 1 scale factor) and encodes all information needed to project 3D world points to 2D image coordinates.

8.4 Camera Calibration (Direct Method)

This section explains how to estimate the projection matrix $\mathbf{P}$ from known correspondences between 3D points and 2D image points.

8.4.1 Problem Formulation

Given n control points $\mathbf{M}_i$ in 3D space and their corresponding projections $\mathbf{m}_i$ in the image, we seek to find the projection matrix $\mathbf{P}$ such that:

$$\mathbf{m}_i \simeq \mathbf{P}\mathbf{M}_i \quad \text{for } i = 1, \ldots, n \tag{14}$$

Since this is an equality up to scale, we can eliminate the scale factor by using the cross product:

$$\mathbf{m}_i \times (\mathbf{P}\mathbf{M}_i) = \mathbf{0} \tag{15}$$

where $\times$ denotes the cross product. This eliminates the scale factor and gives us linear constraints.

8.4.2 Algebraic Tools

Cross Product Formulation:

The cross product $\mathbf{m}_i \times (\mathbf{P}\mathbf{M}_i) = \mathbf{0}$ gives us two independent linear equations (the third is a linear combination of the first two).

For each correspondence, we can write the cross product in matrix form:

$$\begin{pmatrix} 0 & -w_i & v_i \\ w_i & 0 & -u_i \\ -v_i & u_i & 0 \end{pmatrix} \mathbf{P}\mathbf{M}_i = \mathbf{0}$$

where $\mathbf{m}_i = [u_i, v_i, w_i]^T$ is the homogeneous representation of the image point.

Kronecker Product and Vec-Trick:

To transform the matrix equation into a linear system, we use the **vec-trick** (vectorization) and **Kronecker product**:

$$\text{vec}(\mathbf{ABC}) = (\mathbf{C}^T \otimes \mathbf{A})\text{vec}(\mathbf{B}) \tag{16}$$

where $\otimes$ denotes the Kronecker product and $\text{vec}(\cdot)$ vectorizes a matrix by stacking its columns.

Applying this to our problem, we obtain a linear homogeneous system:

$$\mathbf{A} \cdot \text{vec}(\mathbf{P}) = \mathbf{0} \tag{17}$$

where $\mathbf{A}$ is a $2n \times 12$ matrix built from the correspondences, and $\text{vec}(\mathbf{P})$ is the 12-element vector formed by stacking the columns of $\mathbf{P}$.

8.4.3 Solution

The system in Eq. (17) is solved using **Singular Value Decomposition (SVD)**:

1. Compute the SVD of $\mathbf{A}$: $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$
2. The solution is the right singular vector corresponding to the smallest singular value
3. This minimizes the least squares error: $\min \|\mathbf{A} \cdot \text{vec}(\mathbf{P})\|^2$
4. Reshape the solution vector back into a 3×4 matrix $\mathbf{P}$

Calibration Requirements

- Need at least 6 correspondences (each gives 2 equations, need 11 parameters)
- More correspondences improve accuracy and robustness to noise
- Points should be well-distributed in 3D space and image
- The 3D points should not all lie on a plane (non-coplanar configuration)

8.4.4 Calibration Objects

Practical calibration uses objects with known 3D geometry:

- **Checkerboards**: Planar patterns with known square sizes. Easy to detect corners automatically
- **3D calibration objects**: Objects with known 3D coordinates of markers or corners
- **Multiple views**: Capturing the same calibration pattern from different viewpoints increases the number of constraints

8.5 Stereopsis and Triangulation

This section analyzes how to reconstruct 3D structure by observing the scene from two different viewpoints.

8.5.1 Stereopsi

Stereopsis is the process of estimating 3D structure based on **binocular disparity** (the difference in position of the same point in the two images).

Stereopsis Principle

- The same 3D point appears at different positions in the left and right images
- This horizontal difference is called **disparity**
- Disparity is inversely proportional to depth
- By measuring disparity, we can compute the 3D position of points

8.5.2 Main Problems

Stereopsis involves two main problems:

1. **Correspondence (Matching)**: Finding corresponding points between the two images. This is challenging because:
   - The same point may look different due to viewpoint changes
   - Occlusions: some points visible in one image may be hidden in the other
   - Ambiguity: similar-looking points can be confused
2. **Triangulation**: Once correspondences are found, reconstructing the 3D position geometrically

8.5.3 Triangulation (Parallel Cameras)

For the simplified case with **parallel cameras** (aligned cameras with parallel optical axes):

Given a 3D point $\mathbf{M} = [X, Y, Z]^T$:

- Left camera sees it at pixel u
- Right camera sees it at pixel u'
- The disparity is: $d = u' - u$

The depth Z is inversely proportional to the disparity:

$$Z = \frac{b \cdot f}{u' - u} = \frac{b \cdot f}{d} \tag{18}$$

where:

- b is the **baseline** (distance between the two camera centers)
- f is the focal length (assumed equal for both cameras)
- $d = u' - u$ is the disparity

Once Z is known, the X and Y coordinates can be computed from either image:

$$X = \frac{u \cdot Z}{f}, \quad Y = \frac{v \cdot Z}{f}$$

8.5.4 Triangulation (General Case - Linear Eigen)

For the general case with two arbitrary cameras (with known projection matrices $\mathbf{P}$ and $\mathbf{P}'$), a 3D point $\mathbf{M}$ is estimated by solving an overdetermined linear system derived from the projection equations of the two views.

Given:

- Point $\mathbf{m} = [u, v]^T$ in the first image (from camera with matrix $\mathbf{P}$)
- Point $\mathbf{m}' = [u', v']^T$ in the second image (from camera with matrix $\mathbf{P}'$)

The projection equations are:

$$\mathbf{m} \simeq \mathbf{PM}, \quad \mathbf{m}' \simeq \mathbf{P}'\mathbf{M} \tag{19}$$

Using the cross product to eliminate the scale factor:

$$\mathbf{m} \times (\mathbf{PM}) = \mathbf{0}, \quad \mathbf{m}' \times (\mathbf{P}'\mathbf{M}) = \mathbf{0}$$

This gives us 4 linear equations (2 from each view) in 4 unknowns $(X, Y, Z, 1)$. We can write this as:

$$\mathbf{AM} = \mathbf{0} \tag{20}$$

where $\mathbf{A}$ is a 4×4 matrix built from the projection equations.

Least Squares Strategy:

To handle noise in measurements, we use a least squares approach. The solution is found by:

1. Building the system $\mathbf{AM} = \mathbf{0}$ from both views (as in Eq. (20))
2. Computing the SVD: $\mathbf{A} = \mathbf{U\Sigma V}^T$
3. The 3D point $\mathbf{M}$ is the right singular vector corresponding to the smallest singular value
4. This minimizes the reprojection error in a least squares sense

Triangulation Considerations

- **Baseline**: Larger baseline improves depth accuracy but makes correspondence harder
- **Viewing angle**: Points should be visible in both views with sufficient angular separation
- **Noise**: Measurement noise affects the accuracy of the reconstructed 3D point
- **Geometric constraints**: Epipolar geometry can help validate correspondences before triangulation

9 Augmented Reality

Mixed Reality describes the merging of a real-world environment with a computer-generated one. Physical and virtual objects coexist and interact in real time, enabling scenarios such as previewing how a new piece of furniture would look in a room before purchasing it.

Key Concept: Mixed Reality

Successful AR experiences align digital content with the physical world so that virtual objects respond consistently to real-world geometry, lighting, and user motion.

- Synthetic and real objects are visualized within the same scenario.
- Synthetic objects are rendered through a virtual camera controlled by the graphics pipeline.
- Real objects are captured through a physical camera, then processed so they can be displayed within the shared scenario.

Integration between the virtual and real scenes is obtained by calibrating the synthetic camera with the same parameters as the physical one. In practice, both cameras must share the same ***intrinsic*** and ***extrinsic*** characteristics.

9.1 Custom Camera in Unity

Unity is a game engine that allows the creation of 3D games and simulations (render engine). We can create a custom camera in Unity to be used in an AR application, modifying its parameters so they match those of the real camera. Unity exposes both ***intrinsic*** and ***extrinsic*** parameters of the camera, which we derive using calibration techniques.

Unity is specialized in video games and simulations, it is not specialized in AR/VR. Default game objects are:

- Object(s)
- Camera
- Light

We particularly focus on the ***Transform*** component of game objects. The Transform component positions, rotates, and scales each object in the scene. The Camera component exposes many tunable parameters that we can modify to match the real camera.

9.2 Calibration Inputs

- A picture of the real scene.
- The intrinsic and extrinsic parameters of the real camera.
- A synthetic model represented in the same reference system as the real scene (e.g., a 3D model aligned to the captured environment).

We use software such as Zephyr to estimate the camera parameters from the real scene and the synthetic model.

9.2.1 Calibration Objective

The virtual camera in the synthetic scene must be configured with the parameters of the real camera. We therefore control the ***intrinsic*** and ***extrinsic*** parameters of the virtual camera to match the real one. In Unity we mainly need the following ***intrinsic*** values to control the virtual camera:

- Focal length in mm (not in pixels)
- Sensor size (pixel size)
- Lens shift (principal point)

We apply the correct formulas to convert focal length values from pixels to millimeters. In Unity we also set the ***extrinsic*** parameters through the Transform component (position and rotation of the virtual camera). This step is non-trivial because Zephyr and Unity use different reference systems for the extrinsic parameters.

- The x axis is the same in Zephyr and Unity.
- The y axis has opposite direction in Zephyr and Unity.

We reverse the Y axis using the following matrix:

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{21}$$

We switch the Z and Y axes using the following matrix:

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \tag{22}$$

Given the rotation matrix and the translation vector we can compute the extrinsic parameters of the virtual camera. Finally, in Unity the rotation matrix is converted into Euler angles using a dedicated procedure.

After completing these operations we can convert the Zephyr parameters to Unity parameters (Transform component).

Now we overlap the real scene and the synthetic object by creating a canvas as a child of the camera object.

Camera Alignment Checklist

- Match intrinsic parameters (focal length, sensor size, principal point).
- Align extrinsic parameters by mirroring or swapping axes when required.
- Validate the pose by rendering a reference object over the captured frame.

9.3 Our Pipeline

AR Integration Pipeline

1. Create or import the 3D representation of the synthetic object.
2. Calibrate the real camera (e.g., with Zephyr) to obtain intrinsic and extrinsic parameters.
3. Export the calibration data (e.g., XMP file) and convert focal length values from pixels to millimetres.
4. Translate the extrinsic parameters into Unity's Transform conventions (Euler angles and axis adjustments).
5. Render the 3D model with the calibrated virtual camera and composite it with the captured scene.

10 Camera Pose Estimation

Camera pose estimation is the process of estimating the pose (position and orientation) of a camera from a single image or a sequence of images. Given:

- Two cameras observing the same scene.
- The internal set of parameters of both cameras.

- A set of conjugate points in the two images.

Our aim is to etimate the pose of one camera with respect to the other one. We can also consider the problem of motion estimation when rather than using two cameras, we move one camera from one position to another.

Note Once the pose (or motion) has been estimated, we can use it to obtain the full calibration matrix and estimate the 3D structure of the scene (using the triangulation procedure). This process is called **structure from motion** or **structure and motion**.

Note The triangulation component of the camera is estimated up to a scale factor. This is coherent with the principle of Depth-Speed ambiguity. "It is not possible to distinguish between a slower object moving closer to the camera and a faster object moving away from the camera".

10.1 Essential Matrix

Given:

- A camera with known intrinsic parameters $\mathbf{K}$.
- A camera with known intrinsic parameters $\mathbf{K}'$.
- Two images of the same scene captured by the two cameras (or two time instants from the same camera).
- A corresponding set of conjugate points $\mathbf{m}$ and $\mathbf{m}'$ in the two images.

We define the normalized coordinates of the points as:

$$\mathbf{p}_i = \mathbf{K}^{-1}\mathbf{m}_i, \quad \mathbf{p}'_i = \mathbf{K}'^{-1}\mathbf{m}'_i \tag{23}$$

Working with normalized coordinates and fixing the global reference system to the first camera, we can write the projection equations in their canonical form:

$$\mathbf{P} = [\mathbf{I}|\mathbf{0}], \quad \mathbf{P}' = [\mathbf{R}|\mathbf{t}] \tag{24}$$

where $\mathbf{R}$ is the rotation matrix and $\mathbf{t}$ is the translation vector.

To proceed with our estimation, we need to derive the fundamental constraint for normalized coordinates.

10.1.1 Derivation of the Longuet-Higgins Equation

Let us consider two cameras defined by their general projection matrices $\mathbf{P}$ and $\mathbf{P}'$. We can express the decomposition of the camera matrices in terms of rotation matrices $\mathbf{Q}, \mathbf{Q}'$ and translation vectors $\mathbf{d}, \mathbf{d}'$:

$$\mathbf{P} = [\mathbf{Q}|\mathbf{d}], \quad \mathbf{P}' = [\mathbf{Q}'|\mathbf{d}'] \tag{25}$$

The geometric constraint relating corresponding image points $\mathbf{m}$ and $\mathbf{m}'$ (in pixel coordinates) is given by the general epipolar constraint:

$$\mathbf{0} = \mathbf{m}'^T[\mathbf{e}']_\times \mathbf{Q}'\mathbf{Q}^{-1}\mathbf{m} \tag{26}$$

where the term $[\mathbf{e}']_\times$ represents the skew-symmetric matrix of the epipole $\mathbf{e}'$. The epipole is defined by the relative geometry as:

$$\mathbf{e}' = \mathbf{d}' - \mathbf{Q}'\mathbf{Q}^{-1}\mathbf{d}$$

Substituting this into Eq. (26), we identify the core components:

$$\mathbf{0} = \mathbf{m}'^T \underbrace{(\mathbf{d}' - \mathbf{Q}'\mathbf{Q}^{-1}\mathbf{d})}_{\text{Epipole vector } \mathbf{e}'} \times \mathbf{Q}'\mathbf{Q}^{-1}\mathbf{m}$$

To derive the Longuet-Higgins equation, we transition from pixel coordinates to normalized camera coordinates $\mathbf{p}$ and $\mathbf{p}'$, using the intrinsic calibration matrices $\mathbf{K}$ and $\mathbf{K}'$ as defined in Eq. (23):

$$\mathbf{m} = \mathbf{K}\mathbf{p} \quad \text{and} \quad \mathbf{m}' = \mathbf{K}'\mathbf{p}'$$

By defining the relative rotation $\mathbf{R} = \mathbf{Q}'\mathbf{Q}^{-1}$ and the relative translation $\mathbf{t} = \mathbf{e}'$, and substituting the normalized coordinates into the epipolar constraint, meaning $\mathbf{e}' = \mathbf{K}'\mathbf{t}$,

we obtain:

$$\mathbf{0} = \mathbf{m}'^T [\mathbf{K}'\mathbf{t}]_\times \mathbf{K}'\mathbf{R}\mathbf{K}^{-1}\mathbf{m}$$

Substituting $\mathbf{m}$ and $\mathbf{m}'$:

$$\mathbf{0} = (\mathbf{K}'\mathbf{p}')^T [\mathbf{K}'\mathbf{t}]_\times \mathbf{K}'\mathbf{R} \underbrace{\mathbf{K}^{-1}\mathbf{K}}_{\mathbf{I}} \mathbf{p}$$

Which simplifies to:

$$\mathbf{0} = \mathbf{p}'^T \mathbf{K}'^T [\mathbf{K}'\mathbf{t}]_\times \mathbf{K}'\mathbf{R}\mathbf{p} \tag{27}$$

To simplify the term $[\mathbf{K}'\mathbf{t}]_\times$, we recall a fundamental algebraic property of the cross-product matrix. For any invertible matrix $\mathbf{M}$ and vector $\mathbf{v}$, the transformation is given by:

$$[\mathbf{M}\mathbf{v}]_\times = \det(\mathbf{M})\mathbf{M}^{-T}[\mathbf{v}]_\times \mathbf{M}^{-1} \tag{28}$$

Since we are working with homogeneous equations, the scalar determinant factor can be ignored. Setting $\mathbf{M} = \mathbf{K}'$ and $\mathbf{v} = \mathbf{t}$ in Eq. (28), we obtain the projective equivalence:

$$[\mathbf{K}'\mathbf{t}]_\times \simeq \mathbf{K}'^{-T}[\mathbf{t}]_\times \mathbf{K}'^{-1}$$

Substituting this back into Eq. (27):

$$\mathbf{0} = \mathbf{p}'^T \mathbf{K}'^T \left(\mathbf{K}'^{-T}[\mathbf{t}]_\times \mathbf{K}'^{-1}\right) \mathbf{K}'\mathbf{R}\mathbf{p}$$

Grouping the terms highlights the cancellation of the intrinsic matrices:

$$\mathbf{0} = \mathbf{p}'^T \underbrace{\left(\mathbf{K}'^T \mathbf{K}'^{-T}\right)}_{\mathbf{I}} [\mathbf{t}]_\times \underbrace{\left(\mathbf{K}'^{-1}\mathbf{K}'\right)}_{\mathbf{I}} \mathbf{R}\mathbf{p}$$

We finally arrive at the **Longuet-Higgins equation**, which describes the epipolar constraint for calibrated cameras:

Longuet-Higgins Equation

$$\mathbf{p}'^T [\mathbf{t}]_\times \mathbf{R}\mathbf{p} = 0 \tag{29}$$

This is the bilinear-form for conjugate points in normalized coordinates.

Essential Matrix

The matrix $\mathbf{E} = [\mathbf{t}]_\times \mathbf{R}$ is defined as the **Essential Matrix**. This matrix depends on 3 parameters for $\mathbf{R}$ and 2 parameters for $\mathbf{t}$.

$\Longrightarrow$ This equation is homogeneous with respect to $\mathbf{t}$, this reflects the *depth-speed ambiguity*.

10.2 Factorization of the Essential Matrix

We introduce a theorem that gives a characterization of the essential matrix. From this characterization we can factorize the matrix into rotation and translation components.

Factorization Theorem

A real 3×3 matrix $\mathbf{E}$ can be factorized as a product of a non-zero skew-symmetric matrix and a rotation matrix if and only if $\mathbf{E}$ has two equal non-zero singular values and a zero singular value.

Singular Value Decomposition

Given a matrix $\mathbf{A}$ of size $m \times n$, it exists a diagonal matrix $\mathbf{D}$ of size $n \times n$ with positive diagonal elements $\sigma_1 \geq \sigma_2 \geq ... \geq \sigma_n \geq 0$ and two orthogonal matrices $\mathbf{U}$ of size $m \times m$ and $\mathbf{V}$ of size $n \times n$ such that:

$$\mathbf{A} = \mathbf{U}\mathbf{D}\mathbf{V}^T \quad \text{or} \quad \mathbf{U}^T\mathbf{A}\mathbf{V} = \mathbf{D}$$

The columns of $\mathbf{U}$ are the left singular vectors of $\mathbf{A}$ and the columns of $\mathbf{V}$ are the right singular vectors of $\mathbf{A}$. The diagonal elements of $\mathbf{D}$ are the singular values of $\mathbf{A}$.

Proof 1 We need to show that if $\mathbf{E}$ has two equal non-zero singular values and a zero singular value, then it can be factorized as a product of a non-zero skew-symmetric matrix and a rotation matrix.

Let us start with the singular value decomposition of $\mathbf{E}$:

$$\mathbf{E} = \mathbf{U}\mathbf{D}\mathbf{V}^T \tag{30}$$

where $\mathbf{U}$ and $\mathbf{V}$ are orthogonal matrices. Since $\mathbf{E}$ is defined up to scale, the key observation is that the diagonal matrix $\mathbf{D}$ can be decomposed as:

$$\mathbf{D} = \begin{pmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_1 & 0 \\ 0 & 0 & 0 \end{pmatrix} = \underbrace{\begin{pmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}}_{\mathbf{S}'} \underbrace{\begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}}_{\mathbf{R}'} \tag{31}$$

where $\mathbf{S}'$ is a skew-symmetric matrix and $\mathbf{R}'$ is a rotation matrix (representing a rotation of -180 degrees around the z axis).

Substituting this decomposition into Eq. (30):

$$\mathbf{E} = \mathbf{U}\mathbf{S}'\mathbf{R}'\mathbf{V}^T$$

We can insert the identity matrix $\mathbf{U}^T\mathbf{U} = \mathbf{I}$ between $\mathbf{S}'$ and $\mathbf{R}'$:

$$\mathbf{E} = \mathbf{U}\mathbf{S}'\mathbf{U}^T\mathbf{U}\mathbf{R}'\mathbf{V}^T$$

Grouping the terms, we obtain:

$$\mathbf{E} = (\mathbf{U}\mathbf{S}'\mathbf{U}^T)(\mathbf{U}\mathbf{R}'\mathbf{V}^T)$$

Since we are working with homogeneous equations, we can multiply by a scalar factor. We define:

$$\mathbf{E} \simeq \underbrace{(\mathbf{U}\mathbf{S}'\mathbf{U}^T)}_{\mathbf{S}} \underbrace{\det(\mathbf{U}\mathbf{V}^T)(\mathbf{U}\mathbf{R}'\mathbf{V}^T)}_{\mathbf{R}} \tag{32}$$

where $\mathbf{S} = \mathbf{U}\mathbf{S}'\mathbf{U}^T$ and $\mathbf{R} = \det(\mathbf{U}\mathbf{V}^T)(\mathbf{U}\mathbf{R}'\mathbf{V}^T)$.

To complete the proof, we need to verify that $\mathbf{S}$ is a skew-symmetric matrix and $\mathbf{R}$ is a rotation matrix.

First, we show that $\mathbf{S}$ is skew-symmetric by computing its transpose:

$$\mathbf{S}^T = (\mathbf{U}\mathbf{S}'\mathbf{U}^T)^T = \mathbf{U}(\mathbf{S}')^T\mathbf{U}^T = \mathbf{U}(-\mathbf{S}')\mathbf{U}^T = -\mathbf{U}\mathbf{S}'\mathbf{U}^T = -\mathbf{S} \tag{33}$$

where we used the fact that $(\mathbf{S}')^T = -\mathbf{S}'$ since $\mathbf{S}'$ is skew-symmetric.

Next, we verify that $\mathbf{R}$ is a rotation matrix by showing that $\det(\mathbf{R}) = 1$:

$$\det(\mathbf{R}) = \det(\mathbf{U}\mathbf{V}^T)\det(\mathbf{U})\det(\mathbf{R}')\det(\mathbf{V}^T)$$

Since $\mathbf{U}$ and $\mathbf{V}$ are orthogonal matrices, we have $\det(\mathbf{U}) = \pm 1$ and $\det(\mathbf{V}^T) = \pm 1$. Grouping the terms:

$$= \underline{\det(\mathbf{U})\det(\mathbf{V}^\mathbf{T})\det(\mathbf{U})\det(\mathbf{V}^T)}\,\det(\mathbf{R}')$$

which simplifies to:

$$= \underbrace{\det(\mathbf{U})^2}_{=1}\underbrace{\det(\mathbf{V}^T)^2}_{=1}\underbrace{\det(\mathbf{R}')}_{=1} = 1 \tag{34}$$

Therefore, $\mathbf{S}$ is a skew-symmetric matrix and $\mathbf{R}$ is a rotation matrix, completing the proof.

Proof 2 We now prove the converse: if $\mathbf{E} = \mathbf{S}\mathbf{R}$ where $\mathbf{S}$ is a skew-symmetric matrix and $\mathbf{R}$ is a rotation matrix, then $\mathbf{E}$ has two equal non-zero singular values and a zero singular value.

By a well-known theorem, any skew-symmetric matrix can be written in the form $\mathbf{S} \simeq \mathbf{U}\mathbf{S}'\mathbf{U}^T$ where $\mathbf{U}$ is an orthogonal matrix and $\mathbf{S}'$ is a canonical skew-symmetric matrix. Moreover, we have the relation $\mathbf{S}' = \mathrm{Diag}(1, 1, 0)\mathbf{R}'^T$ where $\mathbf{R}'$ is a rotation matrix.

Starting from $\mathbf{E} = \mathbf{S}\mathbf{R}$ and substituting the decomposition of $\mathbf{S}$:

$$\mathbf{E} = \mathbf{S}\mathbf{R} = \mathbf{U}\mathbf{S}'\mathbf{U}^T\mathbf{R}$$

Substituting $\mathbf{S}' = \mathrm{Diag}(1, 1, 0)\mathbf{R}'^T$:

$$\mathbf{E} = \mathbf{U}\mathrm{Diag}(1, 1, 0)\mathbf{R}'^T\mathbf{U}^T\mathbf{R}$$

Rearranging the terms, we can identify the SVD structure:

$$\mathbf{E} = \mathbf{U}\underbrace{\mathrm{Diag}(1, 1, 0)}_{\mathbf{D}}\underbrace{\mathbf{R}'^T\mathbf{U}^T\mathbf{R}}_{\mathbf{V}^T}$$

Since $\mathbf{R}'^T\mathbf{U}^T\mathbf{R}$ is a product of orthogonal matrices, it is itself orthogonal. Therefore:

$$\mathbf{E} = \mathbf{U}\mathbf{D}\mathbf{V}^T \tag{35}$$

This is the SVD of $\mathbf{E}$, where $\mathbf{D} = \mathrm{Diag}(1, 1, 0)$ contains 2 equal non-zero singular values and a zero singular value, as required.

Note The factorization is not unique. It is possible to transpose $\mathbf{S}'$ and $\mathbf{R}'$, leading to 4 different combinations with sign changes:

$$\mathbf{S} = \pm\mathbf{U}\mathbf{S}'\mathbf{U}^T \quad \text{and} \quad \mathbf{R} = \det(\mathbf{U}\mathbf{V}^T)\{\mathbf{R}' \text{ or } \mathbf{R}'^T\}$$

This ambiguity reflects the fact that the essential matrix can be decomposed in multiple equivalent ways, all corresponding to the same geometric configuration up to a sign change.

A flip in the sign of $\mathbf{S}$ changes the right camera to the left camera and vice versa. The choice of $\mathbf{R}$ defines a rotation of 180 degrees around the axis of the camera around the baseline.

10.3 Estimation of the Essential Matrix

Given a set of corresponding points in normalized coordinates, we can estimate the essential matrix $\mathbf{E}$ such that:

$$\mathbf{p_i'}^T \mathbf{E} \mathbf{p}_i = 0 \tag{36}$$

for each corresponding pair $(\mathbf{p}_i, \mathbf{p}_i')$.

This constraint can be rewritten using the vectorization operator:

$$\mathbf{p_i'}^T \mathbf{E} \mathbf{p}_i = 0 \Leftrightarrow \text{vec}(\mathbf{p_i'}^T \mathbf{E} \mathbf{p}_i) = 0$$

Using the Kronecker product property, this becomes:

$$(\mathbf{p_i}^T \otimes \mathbf{p_i'}^T)\text{vec}(\mathbf{E}) = 0$$

Every corresponding pair generates one equation of a homogeneous linear system. Stacking all N correspondences:

$$\underbrace{\begin{bmatrix} \mathbf{p_1}^T \otimes \mathbf{p_1'}^T \\ \vdots \\ \mathbf{p_N}^T \otimes \mathbf{p_N'}^T \end{bmatrix}}_{\mathbf{A}} \underbrace{\text{vec}(\mathbf{E})}_{\mathbf{x}} = \mathbf{0} \tag{37}$$

Since $\mathbf{E}$ is defined up to scale, without loss of generality we can set the last element of $\mathbf{x}$ to 1. This means we just need to solve a system of 8 equations with 8 unknowns rather than 9 (we need 8 points to estimate the essential matrix). In practice we use more than 8 points and the solution is a least squares solution.

Note: Enforcing the Essential Matrix Constraint The matrix estimated with this procedure will not necessarily satisfy the theorem of the essential matrix (i.e., it may not have two equal non-zero singular values and a zero singular value). We can enforce this constraint by defining $\hat{\mathbf{E}}$ from the SVD of $\mathbf{E} = \mathbf{U}\mathbf{D}\mathbf{V}^T$.

Let $\mathbf{D} = \text{diag}(r, s, t)$ with $r \geq s \geq t \geq 0$ be the singular values of $\mathbf{E}$. We define the corrected essential matrix as:

$$\hat{\mathbf{E}} = \mathbf{U}\hat{\mathbf{D}}\mathbf{V}^T \tag{38}$$

where $\hat{\mathbf{D}} = \text{diag}\left(\frac{r+s}{2}, \frac{r+s}{2}, 0\right)$.

It is possible to show that $\hat{\mathbf{E}}$ is the closest matrix to $\mathbf{E}$ (in Frobenius norm) that satisfies the essential matrix theorem. This method is called the **normalized 8-point algorithm**.

10.4 Structure from Motion Algorithm

Structure from Motion Algorithm

Given a set of corresponding points in normalized coordinates and the intrinsic parameters of the cameras, the goal is to estimate:

- The 3D coordinates $\mathbf{M}_i$ of the points
- The motion parameters: rotation matrix $\mathbf{R}$ and translation vector $\mathbf{t}$

The algorithm proceeds through the following steps:

1. **Estimate the essential matrix E** using the 8-point algorithm (or normalized 8-point algorithm) from the set of corresponding points.

2. **Factorize E** into $\mathbf{E} = [\mathbf{t}]_\times \mathbf{R}$ to extract the rotation matrix $\mathbf{R}$ and translation vector $\mathbf{t}$ (up to scale).
3. **Define the projection matrices**:

$$\mathbf{P}_1 = \mathbf{K}_1[\mathbf{I}|\mathbf{0}], \quad \mathbf{P}_2 = \mathbf{K}_2[\mathbf{R}|\mathbf{t}]$$

   where $\mathbf{K}_1$ and $\mathbf{K}_2$ are the intrinsic calibration matrices of the two cameras.
4. **Estimate the 3D points** $\mathbf{M}_i$ by triangulation using the projection matrices $\mathbf{P}_1$ and $\mathbf{P}_2$ and the corresponding image points.

11 Absolute Pose Orientation

The problem of **absolute pose orientation** consists in determining the transformation that aligns two sets of corresponding points in 3D space. Specifically, given:

- A set of points $\{y_i\}_{i=1}^n$ in one coordinate system
- A set of corresponding points $\{X_i\}_{i=1}^n$ in another coordinate system

we seek to find the rotation matrix $\mathbf{R}$, translation vector $\mathbf{t}$, and scale factor s that best align these point sets.

Problem Formulation

We have the following relation:

$$X_i = s(Ry_i + t) \quad \text{for each } i = 1, \ldots, n$$

where:
- $\mathbf{R}$ is the rotation matrix
- $\mathbf{t}$ is the translation vector
- s is the scale factor

Our objective is to solve the following optimization problem:

$$\min_{R,t,s} \sum_{i=1}^{n} ||X_i - s(Ry_i + t)||^2$$

To solve this problem, we employ the **Procrustes method**, which provides an analytical solution by decomposing the problem into simpler subproblems.

11.1 Procrustes Method

The Procrustes method proceeds by systematically isolating the unknown variables. We start from the fundamental equation:

$$X_i = s(Ry_i + t)$$

11.1.1 Isolating the Translation

To isolate the translation vector $\mathbf{t}$, we first compute the mean of both sides of the equation:

$$\frac{1}{n}\sum_{i=1}^{n} X_i = \frac{1}{n}\sum_{i=1}^{n} s(Ry_i + t)$$

Expanding the right-hand side and factoring out the scale factor, we obtain:

$$\frac{1}{n}\sum_{i=1}^{n} X_i = s\frac{1}{n}\sum_{i=1}^{n} Ry_i + st$$

From this, it follows that the translation vector can be expressed as:

$$t = \frac{1}{s}\left(\frac{1}{n}\sum_{i=1}^{n} X_i - s\frac{1}{n}\sum_{i=1}^{n} Ry_i\right)$$

Substituting this expression back into the original equation for X_i, we get:

$$X_i = s\left(Ry_i + \frac{1}{s}\left(\frac{1}{n}\sum_{i=1}^{n} X_i - s\frac{1}{n}\sum_{i=1}^{n} Ry_i\right)\right)$$

Expanding and rearranging terms, we arrive at:

$$X_i = sRy_i + \frac{1}{n}\sum_{i=1}^{n} X_i - s\frac{1}{n}\sum_{i=1}^{n} Ry_i$$

By grouping terms and introducing centered coordinates, we obtain:

$$X_i - \underbrace{\frac{1}{n}\sum_{i=1}^{n} X_i}_{\bar{X}} = sR\left(y_i - \underbrace{\frac{1}{n}\sum_{i=1}^{n} y_i}_{\bar{y}}\right)$$

where $\bar{X}$ and $\bar{y}$ denote the centroids of the respective point sets.

11.1.2 Centered Coordinates

Defining the centered coordinates as:

$$\hat{X}_i = X_i - \bar{X} = X_i - \frac{1}{n}\sum_{i=1}^{n} X_i$$
$$\hat{y}_i = y_i - \bar{y} = y_i - \frac{1}{n}\sum_{i=1}^{n} y_i$$

we obtain the simplified equation:

$$\hat{X}_i = sR\hat{y}_i$$

Centered Coordinates

By centering both point sets at their respective centroids, we eliminate the translation component from the problem. This reduces the original problem to finding only the rotation matrix $\mathbf{R}$ and scale factor s, which significantly simplifies the optimization.

11.1.3 Estimating the Scale Factor

To isolate the scale factor s, we exploit the algebraic properties of the equation. Since an orthogonal matrix preserves vector lengths, we can write:

$$||\hat{X}_i|| = ||sR\hat{y}_i||$$

Since s is a scalar, we can factor it out:

$$||\hat{X}_i|| = |s|\,||R\hat{y}_i||$$

Furthermore, since $\mathbf{R}$ is an orthogonal matrix and does not change the length of a vector, we have:

$$||\hat{X}_i|| = |s|\,||\hat{y}_i||$$

Therefore, the scale factor can be computed as:

$$|s| = \frac{||\hat{X}_i||}{||\hat{y}_i||}$$

In practice, to obtain a more robust estimate, we typically compute the scale factor using all points:

$$s = \frac{\sum_{i=1}^{n} ||\hat{X}_i||}{\sum_{i=1}^{n} ||\hat{y}_i||}$$

11.1.4 Estimating the Rotation Matrix

Now we need to estimate the rotation matrix $\mathbf{R}$. To do this efficiently, we adopt a matrix notation by stacking all centered coordinates into matrices:

$$\hat{\mathbf{X}}^{3\times n} = \begin{bmatrix} | & | & & | \\ \hat{\mathbf{x}}_1 & \hat{\mathbf{x}}_2 & \dots & \hat{\mathbf{x}}_n \\ | & | & & | \end{bmatrix}$$

$$\hat{\mathbf{Y}}^{3\times n} = \begin{bmatrix} | & | & & | \\ \hat{\mathbf{y}}_1 & \hat{\mathbf{y}}_2 & \dots & \hat{\mathbf{y}}_n \\ | & | & & | \end{bmatrix}$$

These matrices are obtained by concatenating the centered vectors $\hat{\mathbf{X}}_i$ and $\hat{\mathbf{y}}_i$, respectively.

Using this matrix notation, we can rewrite the optimization problem as:

$$\min_R \sum_{i=1}^{n} ||\hat{X}_i - sR\hat{y}_i||^2$$

This is equivalent to minimizing the Frobenius norm:

$$||\hat{\mathbf{X}} - s\mathbf{R}\hat{\mathbf{Y}}||_F^2$$

Since we have already estimated the scale factor s, we can normalize the problem by dividing both sides by s, leading to:

$$\min_R ||\hat{\mathbf{X}} - \mathbf{R}\hat{\mathbf{Y}}||_F^2$$

This formulation corresponds to a well-known problem in linear algebra: the **orthogonal Procrustes problem**.

11.2 Orthogonal Procrustes Problem

Orthogonal Procrustes Problem

Given two matrices $\mathbf{A}$ and $\mathbf{B}$ of the same dimensions, we want to find the orthogonal matrix $\mathbf{W}$ that minimizes the Frobenius norm of the difference between $\mathbf{A}$ and $\mathbf{WB}$:

$$\min_{\mathbf{W}} ||\mathbf{A} - \mathbf{WB}||_F^2$$

subject to the constraint that $\mathbf{W}$ is an orthogonal matrix, meaning $\mathbf{W}^T\mathbf{W} = \mathbf{I}$.

This problem has a unique solution that can be obtained through the **Singular Value Decomposition (SVD)**. Specifically, if we compute the SVD of $\mathbf{BA}^T$:

$$\mathbf{BA}^T = \mathbf{UDV}^T$$

then the optimal orthogonal matrix is given by:

$$\mathbf{W} = \mathbf{VU}^T$$

11.2.1 Ensuring Proper Rotation

In our specific case, $\mathbf{R}$ is not just an orthogonal matrix, but it must be a **rotation matrix**, which requires that $\det(\mathbf{R}) = 1$ (i.e., it preserves orientation). The solution $\mathbf{VU}^T$ obtained from the SVD may have determinant -1, which would correspond to a reflection rather than a rotation.

To guarantee a proper rotation, we modify the solution as follows:

$$\mathbf{R} = \mathbf{V} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & \det(\mathbf{VU}^T) \end{bmatrix} \mathbf{U}^T$$

where the SVD is computed as:

$$\mathbf{UDV}^T = \hat{\mathbf{Y}}\hat{\mathbf{X}}^T$$

Rotation vs. Reflection

The modification ensures that the determinant of $\mathbf{R}$ is exactly 1, guaranteeing that the transformation represents a proper rotation rather than a reflection. This is crucial for applications where orientation must be preserved.

11.2.2 Final Solution

Once we have computed the rotation matrix $\mathbf{R}$ and the scale factor s, we can substitute them back into the equation for the translation vector $\mathbf{t}$:

$$\mathbf{t} = \frac{1}{s}\left(\frac{1}{n}\sum_{i=1}^{n}\mathbf{X}_i - s\mathbf{R}\frac{1}{n}\sum_{i=1}^{n}\mathbf{y}_i\right) = \bar{\mathbf{X}} - s\mathbf{R}\bar{\mathbf{y}}$$

where $\bar{\mathbf{X}}$ and $\bar{\mathbf{y}}$ are the centroids of the respective point sets.

This completes the solution to the absolute pose orientation problem, providing estimates for all three unknown parameters: rotation $\mathbf{R}$, translation $\mathbf{t}$, and scale s.